GRAMATICA .FORM

LEXER

```
/* ── Definiciones ── */
DIGITO      = [0-9]
LETTER      = [a-zA-Z]
SEPARADORES = [ \n\r\t]+
ENTERO      = {DIGITO}+
DECIMAL     = {DIGITO}+ \. {DIGITO}+
ID          = {LETTER} ({LETTER} | {DIGITO} | "_")*
COMENTARIO  = "$" [^\r\n]*
HEXCOLOR    = \# [0-9a-fA-F]{6}

%state STRING_STATE
%state COMMENT_BLOCK

%%

/* ═══════════════════════════════════════════════
   ESTADO INICIAL
   ═══════════════════════════════════════════════ */
<YYINITIAL> {

    /* ── Comentarios ── */
    {COMENTARIO}           { /* ignorar */ }
    "/*"                   { yybegin(COMMENT_BLOCK); }

    /* ── Espacios ── */
    {SEPARADORES}          { /* ignorar */ }

    /*
     * ── STRINGS ──
     * La comilla SIEMPRE va al STRING_STATE.
     * Las claves de estilo como "color", "font family" etc.
     * se reconocen DENTRO del STRING_STATE como casos especiales
     * y retornan su token correspondiente.
     */
    \"                     {
        stringBuffer.setLength(0);
        yybegin(STRING_STATE);
    }

    /* ── Variables ── */
    "number"               { return symbol(sym.VARIABLENUMBER); }
    "string"               { return symbol(sym.VARIABLESTRING); }
    "special"              { return symbol(sym.SPECIAL);        }

    /* ── Preguntas ── */
    "OPEN_QUESTION"        { return symbol(sym.OPENQUESTION);     }
    "DROP_QUESTION"        { return symbol(sym.DROPQUESTION);     }
    "SELECT_QUESTION"      { return symbol(sym.SELECTQUESTION);   }
    "MULTIPLE_QUESTION"    { return symbol(sym.MULTIPLEQUESTION); }
```

```
/* — Elementos — */
"SECTION"          { return symbol(sym.SECTION);  }
"TABLE"            { return symbol(sym.TABLE);    }
"TEXT"             { return symbol(sym.TEXT);     }
"draw"             { return symbol(sym.DRAW);     }

/* — Atributos — */
"elements"         { return symbol(sym.ELEMENTS);    }
"styles"           { return symbol(sym.STYLES);      }
"label"            { return symbol(sym.LABEL);       }
"content"          { return symbol(sym.CONTENT);     }
"width"            { return symbol(sym.WIDTH);       }
"height"           { return symbol(sym.HEIGHT);      }
"pointX"           { return symbol(sym.POINTX);      }
"pointY"           { return symbol(sym.POINTY);      }
"options"          { return symbol(sym.OPTIONS);     }
"correct"          { return symbol(sym.CORRECT);     }
"orientation"      { return symbol(sym.ORIENTATION); }

/* — Estilos tipográficos — */
"SOLID"            { return symbol(sym.SOLID);      }
"DOUBLE"           { return symbol(sym.DOUBLE);     }
"DOTTED"           { return symbol(sym.DOTTED);     }
"LINE"             { return symbol(sym.LINE);       }
"MONO"             { return symbol(sym.MONO);       }
"SANS_SERIF"       { return symbol(sym.SANS_SERIF); }
"CURSIVE"          { return symbol(sym.CURSIVE);    }

/* — Orientación — */
"VERTICAL"         { return symbol(sym.VERTICAL);   }
"HORIZONTAL"       { return symbol(sym.HORIZONTAL); }

/* — Colores base — */
"RED"              { return symbol(sym.RED);    }
"BLUE"             { return symbol(sym.BLUE);   }
"GREEN"            { return symbol(sym.GREEN);  }
"PURPLE"           { return symbol(sym.PURPLE); }
"SKY"              { return symbol(sym.SKY);    }
"YELLOW"           { return symbol(sym.YELLOW); }
"BLACK"            { return symbol(sym.BLACK);  }
"WHITE"            { return symbol(sym.WHITE);  }

/* — Bloques de código — */
"IF"               { return symbol(sym.IF);    }
"ELSE"             { return symbol(sym.ELSE);  }
"WHILE"            { return symbol(sym.WHILE); }
"DO"               { return symbol(sym.DO);    }
"FOR"              { return symbol(sym.FOR);   }
"in"               { return symbol(sym.IN);    }

/* — PokeAPI — */
"who_is_that_pokemon"   { return symbol(sym.WHO_IS_THAT_POKEMON); }
```

```
/* ── Hex color ── */
{HEXCOLOR}            { return symbol(sym.HEXCOLOR, yytext()); }

/* ── RGB completo como un solo token — va ANTES que PARENTESIS_ABRE ── */
"(" {DIGITO}+ "," {DIGITO}+ "," {DIGITO}+ ")"
   {
      return symbol(sym.RGB_COLOR, yytext());
   }

/* ── HSL completo como un solo token ── */
"<" {DIGITO}+ ("." {DIGITO}+)? "," {DIGITO}+ ("." {DIGITO}+)? "," {DIGITO}+ ("." {DIGITO}+)? ">"
   {
      return symbol(sym.HSL_COLOR, yytext());
   }

/* ── Operadores ── */
"=="   { return symbol(sym.IGUALIGUAL); }
"!!"   { return symbol(sym.DIFERENTE);  }
">="   { return symbol(sym.MAYORIGUAL); }
"<="   { return symbol(sym.MENORIGUAL); }
"&&"   { return symbol(sym.AND);       }
"||"   { return symbol(sym.OR);        }
"~"    { return symbol(sym.NOT);       }
"="    { return symbol(sym.IGUAL);     }
">"    { return symbol(sym.MAYOR);     }
"<"    { return symbol(sym.MENOR);     }
"+"    { return symbol(sym.SUMA);      }
"-"    { return symbol(sym.RESTA);     }
"*"    { return symbol(sym.MULTI);     }
"/"    { return symbol(sym.DIVISION);  }
"^"    { return symbol(sym.POTENCIA);  }
"%"    { return symbol(sym.MODULO);    }
".."   { return symbol(sym.RANGO);     }

/* ── Delimitadores ── */
"("    { return symbol(sym.PARENTESIS_ABRE);   }
")"    { return symbol(sym.PARENTESIS_CIERRA); }
"["    { return symbol(sym.CORCHETE_ABRE);     }
"]"    { return symbol(sym.CORCHETE_CIERRA);   }
"{"    { return symbol(sym.LLAVE_ABRE);        }
"}"    { return symbol(sym.LLAVE_CIERRA);      }
";"    { return symbol(sym.PUNTOCOMA);         }
","    { return symbol(sym.COMA);              }
":"    { return symbol(sym.DOSPUNTOS);         }
"?"    { return symbol(sym.INTERROGACION);     }
"."    { return symbol(sym.PUNTO);             }

/* ── Números ── */
{DECIMAL}   { return symbol(sym.DECIMAL, yytext()); }
{ENTERO}    { return symbol(sym.ENTERO,  yytext()); }

/* ── Identificadores ── */
```

```
  {ID}       { return symbol(sym.VARIABLE, yytext()); }

  /* ── Error ── */
  .          { errorLexer(yytext()); }
}

/* ═══════════════════════════════════════════════
  ESTADO STRING_STATE
  ═══════════════════════════════════════════════ */
<STRING_STATE> {

  /* ── Cierre del string ── */
  \"  {
     yybegin(YYINITIAL);
     String resultado = stringBuffer.toString();
     switch (resultado) {
        case "color":           return symbol(sym.STYLE_COLOR);
        case "background color": return symbol(sym.STYLE_BACKGROUND);
        case "font family":     return symbol(sym.STYLE_FONT);
        case "text size":       return symbol(sym.STYLE_SIZE);
        case "border":          return symbol(sym.STYLE_BORDER);
        default:
           return symbol(sym.CADENA, resultado);
     }
  }

  /* ── Emojis ── */
  "@[:smile:]"              { stringBuffer.append("😀"); }
  "@[:sad:]"                { stringBuffer.append("😢"); }
  "@[:serious:]"            { stringBuffer.append("😐"); }
  "@[:heart:]"              { stringBuffer.append("❤"); }
  "@[:cat:]"                { stringBuffer.append("😺"); }
  "@[:^^:]"                 { stringBuffer.append("😺"); }
  "@[:star:]"               { stringBuffer.append("⭐"); }

  "@[:star:" {DIGITO}+ ":]"   {
     String txt   = yytext();
     int    start = txt.indexOf("star:") + 5;
     int    end   = txt.lastIndexOf(":");
     int    n     = Integer.parseInt(txt.substring(start, end));
     for (int i = 0; i < n; i++) stringBuffer.append("⭐");
  }

  "@[:star-" {DIGITO}+ "]"    {
     String txt   = yytext();
     int    start = txt.indexOf("star-") + 5;
     int    end   = txt.lastIndexOf("]");
     int    n     = Integer.parseInt(txt.substring(start, end));
     for (int i = 0; i < n; i++) stringBuffer.append("⭐");
  }

  "@[" "<"+ "3"+ "]"         { stringBuffer.append("❤"); }
  "@[:" ")"+  "]"            { stringBuffer.append("😀"); }
```

```
  "@[:" "("+ "]"            { stringBuffer.append("🥲"); }
  "@[:" "|"+ "]"            { stringBuffer.append("😐"); }
  "@"                       { stringBuffer.append('@');  }

  /* ── Signos de interrogación españoles — se guardan como texto ── */
  /* ¿ y ? dentro de strings son texto normal, NO comodines         */
  "¿"                       { stringBuffer.append('¿'); }

  /*
   * CLAVE: el ? dentro de un string se guarda como el carácter
   * Unicode \u003F (representación interna), NO como "?"
   * Así contieneComodin("¿Cuántos años tienes\u003F") != true
   * porque buscamos exactamente "?" y no \u003F
   *
   * Usamos un marcador especial que no sea "?" para que
   * contieneComodin no lo detecte como comodín.
   */


  /* ── ¿ y ? dentro de strings — se guardan como marcador \uFFFE ── */
  "¿"   { stringBuffer.append('¿');    }
   "?"   { stringBuffer.append('\uFFFE'); }  /* ← marcador temporal, NO es "?" */

   /* ── Escape ── */
   "\\\""   { stringBuffer.append('"');  }
   "\\" .   { stringBuffer.append(yytext()); }

   /* ── Contenido normal — ahora excluye ? y ¿ explícitamente ── */
   [^\"\\\n\r@¿?]+   { stringBuffer.append(yytext()); }

  \n | \r | \r\n    {
      yybegin(YYINITIAL);
      String res = stringBuffer.toString().replace('\uFFFE', '?');
      return symbol(sym.CADENA, res);
  }
}
/* ═══════════════════════════════════════════════════
  ESTADO COMMENT_BLOCK  /* ... */
  ═══════════════════════════════════════════════════ */
<COMMENT_BLOCK> {
  "*/"       { yybegin(YYINITIAL); }
  [^*]+      { /* ignorar */ }
  "*"        { /* ignorar */ }
}
```

PARSER

```
/* ============================================================
  TERMINALES
  ============================================================ */
terminal SECTION, TABLE, TEXT;
terminal OPENQUESTION, DROPQUESTION, SELECTQUESTION, MULTIPLEQUESTION;
terminal VARIABLENUMBER, VARIABLESTRING;
terminal SPECIAL, DRAW, PUNTO, INTERROGACION;
terminal IGUALIGUAL, DIFERENTE, MAYORIGUAL, MENORIGUAL;
terminal AND, OR, NOT;
terminal IGUAL, MAYOR, MENOR;
terminal SUMA, RESTA, MULTI, DIVISION, POTENCIA, MODULO;
terminal PARENTESIS_ABRE, PARENTESIS_CIERRA;
terminal CORCHETE_ABRE,   CORCHETE_CIERRA;
terminal LLAVE_ABRE,      LLAVE_CIERRA;
terminal PUNTOCOMA, COMA, DOSPUNTOS;
terminal DECIMAL, ENTERO;
terminal VARIABLE, CADENA;
terminal ELEMENTS, STYLES;
terminal WIDTH, HEIGHT, POINTX, POINTY, ORIENTATION;
terminal SOLID, DOUBLE, DOTTED, LINE;
terminal MONO, SANS_SERIF, CURSIVE;
terminal LABEL, OPTIONS, CORRECT, CONTENT;
terminal VERTICAL, HORIZONTAL;
terminal HEXCOLOR;
terminal RED, BLUE, GREEN, PURPLE, SKY, YELLOW, BLACK, WHITE;
terminal STYLE_COLOR, STYLE_BACKGROUND, STYLE_FONT, STYLE_SIZE, STYLE_BORDER;
terminal IF, ELSE, WHILE, DO, FOR, IN, RANGO;
terminal WHO_IS_THAT_POKEMON;
terminal String RGB_COLOR;
terminal String HSL_COLOR;
/*
  NO TERMINALES
   */
non terminal inicio;
non terminal lista_elementos;
non terminal Object element_raiz;
non terminal Object element;

/* Variables */
non terminal declaracion;
non terminal asignacion;
non terminal Object expresion;
non terminal Object condicion;

/* Preguntas */
non terminal OpenQuestion          open_q;
non terminal DropQuestion          drop_q;
non terminal SelectQuestion        select_q;
non terminal MultipleQuestion      multiple_q;
```

```
non terminal ArrayList<PropertyItem> question_props;
non terminal PropertyItem           question_prop;
non terminal ArrayList<String>      lista_opciones;
non terminal ArrayList<String>      lista_strings;
non terminal ArrayList<Integer>     int_list;

/* Texto */
non terminal Object                 text_decl;
non terminal ArrayList<PropertyItem> text_props;
non terminal PropertyItem           text_prop;

/* Secciones */
non terminal Object   section;
non terminal ArrayList section_props;
non terminal Object    section_prop;
non terminal ArrayList element_list;

/* Tablas */
non terminal Object   table;
non terminal ArrayList table_props;
non terminal PropertyItem table_prop;
non terminal ArrayList table_rows;
non terminal ArrayList table_row;
non terminal ArrayList table_cells;

/* Estilos */
non terminal ColorValue       color_value;
non terminal Object           style_item;
non terminal ArrayList<Object> styles_block;
non terminal ArrayList<Object> style_list;
non terminal String           font_value;

/* Variables especiales */
non terminal                  special_decl;
non terminal ArrayList<PropertyItem> special_props;
non terminal PropertyItem           special_prop;
non terminal Object                 special_prop_value;
non terminal                  draw_stmt;
non terminal ArrayList<Object>      parametros;
non terminal String                 tipo_pregunta;

/* Bloques de código */
non terminal Object           bloque_codigo;
non terminal ArrayList<Object> lista_instrucciones_bloque;
non terminal Object           instruccion_bloque;
non terminal Object           bloque_if;
non terminal Object           bloque_while;
non terminal Object           bloque_do_while;
non terminal Object           bloque_for;

non terminal Object element_en_lista;
non terminal String tipo_borde;
non terminal String comodin;
```

```
non terminal Object comodin_valor;
non terminal Object comodin_color;
/* PokeAPI */
non terminal ArrayList<String> poke_call;

/* ═══════════════════════════════════════════════════
  PRECEDENCIA
  ═══════════════════════════════════════════════════ */
precedence left OR;
precedence left AND;
precedence right NOT;
precedence left MAYOR, MENOR, IGUALIGUAL, DIFERENTE, MAYORIGUAL, MENORIGUAL;
precedence left SUMA, RESTA;
precedence left MULTI, DIVISION, MODULO;
precedence right POTENCIA;

start with inicio;

/*
  INICIO
   */
inicio ::= lista_elementos;

lista_elementos ::=
   lista_elementos element_raiz:e
   {:
      if (e != null) ResultParser.currentForm.addElement(e);
   :}
   | element_raiz:e
   {:
      if (e != null) ResultParser.currentForm.addElement(e);
   :}
   ;

/* Nivel inicial */
element_raiz ::=
   element:e      {: RESULT = e; :}
   | declaracion   {: RESULT = null; :}
   | asignacion    {: RESULT = null; :}
   | special_decl  {: RESULT = null; :}
   | draw_stmt     {: RESULT = null; :}
   | bloque_codigo {: RESULT = null; :}
   | error PUNTOCOMA {: RESULT = null; :}
   ;

/* Dentro de elements:{} — no acepta declaraciones ni especiales */
element ::=
   section:s     {: RESULT = s; :}
   | table:t     {: RESULT = t; :}
   | text_decl:x  {: RESULT = x; :}
   | open_q:o     {: RESULT = o; :}
   | drop_q:d     {: RESULT = d; :}
   | select_q:s   {: RESULT = s; :}
```

```
    | multiple_q:m {: RESULT = m; :}
    ;

/*
  DECLARACIONES Y ASIGNACIONES
  */
declaracion ::=
    VARIABLENUMBER VARIABLE:v IGUAL expresion:e
    {:
        parser.semantico.declararVariable(v.toString(), "number", e, 0, 0);
        ResultParser.currentForm.addVariable(
            new VariableModel("number", v.toString(), e));
    :}
    | VARIABLENUMBER VARIABLE:v
    {:
        parser.semantico.declararVariable(v.toString(), "number", 0.0, 0, 0);
        ResultParser.currentForm.addVariable(
            new VariableModel("number", v.toString(), 0.0));
    :}
    | VARIABLESTRING VARIABLE:v IGUAL expresion:e
    {:
        parser.semantico.declararVariable(v.toString(), "string", e, 0, 0);
        ResultParser.currentForm.addVariable(
            new VariableModel("string", v.toString(), e));
    :}
    | VARIABLESTRING VARIABLE:v
    {:
        parser.semantico.declararVariable(v.toString(), "string", "", 0, 0);
        ResultParser.currentForm.addVariable(
            new VariableModel("string", v.toString(), ""));
    :}

    ;

asignacion ::=
    VARIABLE:v IGUAL expresion:e
    {:
        parser.semantico.asignarVariable(v.toString(), e, 0, 0);
    :}
    ;

/* ═══════════════════════════════════════════════
  EXPRESIONES
  ═══════════════════════════════════════════════ */

comodin ::=
    INTERROGACION
    {:
        RESULT = "?";
    :}
;

expresion ::=
```

```
    ENTERO:e
      {: RESULT = Double.parseDouble(e.toString()); :}

   | DECIMAL:d
      {: RESULT = Double.parseDouble(d.toString()); :}

   | CADENA:c
      {: RESULT = c.toString(); :}

   | VARIABLE:v
      {:
         Object valor = parser.semantico.getValorVariable(v.toString());
         RESULT = (valor != null) ? valor : v.toString();
      :}

   | comodin:c
      {: RESULT = "?"; :}

   | expresion:a SUMA expresion:b
      {: RESULT = parser.semantico.suma(a, b); :}

   | expresion:a RESTA expresion:b
      {: RESULT = parser.semantico.resta(a, b); :}

   | expresion:a MULTI expresion:b
      {: RESULT = parser.semantico.multiplicacion(a, b); :}

   | expresion:a DIVISION expresion:b
      {: RESULT = parser.semantico.division(a, b, 0, 0); :}

   | expresion:a POTENCIA expresion:b
      {: RESULT = parser.semantico.potencia(a, b); :}

   | expresion:a MODULO expresion:b
      {: RESULT = parser.semantico.modulo(a, b, 0, 0); :}

   | PARENTESIS_ABRE expresion:e PARENTESIS_CIERRA
      {: RESULT = e; :}

   | RESTA expresion:e
      {: RESULT = -parser.semantico.evaluar(e); :}

   | SUMA expresion:e
      {: RESULT = +parser.semantico.evaluar(e); :}
;


/*
  CONDICIONES
   */
condicion ::=
   expresion:a MAYOR expresion:b
```

```
  {:
      parser.semantico.resetOperadorLogico();
      RESULT = parser.semantico.mayor(a, b);
  :}
  | expresion:a MENOR expresion:b
  {:
      parser.semantico.resetOperadorLogico();
      RESULT = parser.semantico.menor(a, b);
  :}
  | expresion:a IGUALIGUAL expresion:b
  {:
      parser.semantico.resetOperadorLogico();
      RESULT = parser.semantico.igual(a, b);
  :}
  | expresion:a DIFERENTE expresion:b
  {:
      parser.semantico.resetOperadorLogico();
      RESULT = parser.semantico.diferente(a, b);
  :}
  | expresion:a MAYORIGUAL expresion:b
  {:
      parser.semantico.resetOperadorLogico();
      RESULT = parser.semantico.mayorIgual(a, b);
  :}
  | expresion:a MENORIGUAL expresion:b
  {:
      parser.semantico.resetOperadorLogico();
      RESULT = parser.semantico.menorIgual(a, b);
  :}
  | condicion:a AND condicion:b
  {: RESULT = parser.semantico.and(a, b); :}
  | condicion:a OR condicion:b
  {: RESULT = parser.semantico.or(a, b); :}
  | NOT condicion:c
  {: RESULT = parser.semantico.not(c); :}
  | PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  {: RESULT = c; :}
  ;

/*
  BLOQUES DE CÓDIGO
  */
bloque_codigo ::=
  bloque_if:b        {: RESULT = b; :}
  | bloque_while:b    {: RESULT = b; :}
  | bloque_do_while:b {: RESULT = b; :}
  | bloque_for:b      {: RESULT = b; :}
  ;

lista_instrucciones_bloque ::=
  lista_instrucciones_bloque:l instruccion_bloque:e
  {:
      if (l == null) l = new ArrayList<Object>();
```

```
        if (e != null) l.add(e);
        RESULT = l;
    :}
    | instruccion_bloque:e
    {:
        ArrayList<Object> list = new ArrayList<Object>();
        if (e != null) list.add(e);
        RESULT = list;
    :}
    ;

instruccion_bloque ::=
    element_en_lista:e  {: RESULT = e; :}
    | declaracion       {: RESULT = null; :}
    | asignacion        {: RESULT = null; :}
    | bloque_codigo     {: RESULT = null; :}
    | error PUNTOCOMA   {: RESULT = null; :}
    /* Recuperación sin ; — para asignaciones sin punto y coma */
    | error LLAVE_CIERRA
    {:

        RESULT = null;
    :}
    ;

/*
  CONDICIONAL IF
    .
    */
bloque_if ::=

    /* IF vacío */
    IF PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
    LLAVE_ABRE LLAVE_CIERRA
    {: RESULT = null; :}

    /* IF con contenido */
    | IF PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
      LLAVE_ABRE lista_instrucciones_bloque:si LLAVE_CIERRA
    {:
        if (parser.semantico.evaluarCondicion(c))
            for (Object o : (ArrayList<Object>) si)
                if (o != null) ResultParser.currentForm.addElement(o);
        RESULT = null;
    :}

    /* IF vacío + ELSE vacío */
    | IF PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
      LLAVE_ABRE LLAVE_CIERRA
      ELSE LLAVE_ABRE LLAVE_CIERRA
    {: RESULT = null; :}

    /* IF + ELSE vacío */
```

```
| IF PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  LLAVE_ABRE lista_instrucciones_bloque:si LLAVE_CIERRA
  ELSE LLAVE_ABRE LLAVE_CIERRA
{:
    if (parser.semantico.evaluarCondicion(c))
        for (Object o : (ArrayList<Object>) si)
            if (o != null) ResultParser.currentForm.addElement(o);
    RESULT = null;
:}

/* IF vacío + ELSE con contenido */
| IF PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  LLAVE_ABRE LLAVE_CIERRA
  ELSE LLAVE_ABRE lista_instrucciones_bloque:no LLAVE_CIERRA
{:
    if (!parser.semantico.evaluarCondicion(c))
        for (Object o : (ArrayList<Object>) no)
            if (o != null) ResultParser.currentForm.addElement(o);
    RESULT = null;
:}

/* IF + ELSE */
| IF PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  LLAVE_ABRE lista_instrucciones_bloque:si LLAVE_CIERRA
  ELSE LLAVE_ABRE lista_instrucciones_bloque:no LLAVE_CIERRA
{:
    ArrayList<Object> rama = parser.semantico.evaluarCondicion(c)
        ? (ArrayList<Object>) si : (ArrayList<Object>) no;
    for (Object o : rama)
        if (o != null) ResultParser.currentForm.addElement(o);
    RESULT = null;
:}

/* IF + ELSE IF (sin ELSE final) */
| IF PARENTESIS_ABRE condicion:c1 PARENTESIS_CIERRA
  LLAVE_ABRE lista_instrucciones_bloque:si LLAVE_CIERRA
  ELSE IF PARENTESIS_ABRE condicion:c2 PARENTESIS_CIERRA
  LLAVE_ABRE lista_instrucciones_bloque:elseif LLAVE_CIERRA
{:
    if (parser.semantico.evaluarCondicion(c1)) {
        for (Object o : (ArrayList<Object>) si)
            if (o != null) ResultParser.currentForm.addElement(o);
    } else if (parser.semantico.evaluarCondicion(c2)) {
        for (Object o : (ArrayList<Object>) elseif)
            if (o != null) ResultParser.currentForm.addElement(o);
    }
    RESULT = null;
:}

/* IF + ELSE IF + ELSE */
| IF PARENTESIS_ABRE condicion:c1 PARENTESIS_CIERRA
  LLAVE_ABRE lista_instrucciones_bloque:si LLAVE_CIERRA
  ELSE IF PARENTESIS_ABRE condicion:c2 PARENTESIS_CIERRA
```

```
    LLAVE_ABRE lista_instrucciones_bloque:elseif LLAVE_CIERRA
    ELSE LLAVE_ABRE lista_instrucciones_bloque:no LLAVE_CIERRA
  {:
      if (parser.semantico.evaluarCondicion(c1)) {
          for (Object o : (ArrayList<Object>) si)
              if (o != null) ResultParser.currentForm.addElement(o);
      } else if (parser.semantico.evaluarCondicion(c2)) {
          for (Object o : (ArrayList<Object>) elseif)
              if (o != null) ResultParser.currentForm.addElement(o);
      } else {
          for (Object o : (ArrayList<Object>) no)
              if (o != null) ResultParser.currentForm.addElement(o);
      }
      RESULT = null;
  :}
  ;

/*
  WHILE

  */
bloque_while ::=

  /* WHILE vacío */
  WHILE PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  LLAVE_ABRE LLAVE_CIERRA
  {: RESULT = null; :}

  /* WHILE con contenido */
  | WHILE PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
    LLAVE_ABRE lista_instrucciones_bloque:instr LLAVE_CIERRA
  {:
      int iter = 0;

          condicion = parser.semantico.recalcularUltimaCondicion();
      }
      parser.semantico.reportarLimiteCiclo("WHILE", iter, 0, 0);
      RESULT = null;
  :}
  ;

/*
  DO-WHILE
  Garantiza al menos una ejecución.
  */
bloque_do_while ::=

  /* DO-WHILE vacío */
  DO LLAVE_ABRE LLAVE_CIERRA
  WHILE PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  {: RESULT = null; :}

  /* DO-WHILE con contenido */
```

```
  | DO LLAVE_ABRE lista_instrucciones_bloque:instr LLAVE_CIERRA
   WHILE PARENTESIS_ABRE condicion:c PARENTESIS_CIERRA
  {:
     int iter = 0;
     do {
        for (Object o : (ArrayList<Object>) instr)
           if (o != null) ResultParser.currentForm.addElement(o);
        iter++;
        /* Después de la primera iteración re-evalúa dinámicamente */
     } while (parser.semantico.recalcularUltimaCondicion()
              && iter < parser.semantico.getMaxIteraciones());
     parser.semantico.reportarLimiteCiclo("DO-WHILE", iter, 0, 0);
     RESULT = null;
  :}
  ;

/* ═══════════════════════════════════════════════════
  FOR CLÁSICO y FOR con RANGO
  ═══════════════════════════════════════════════════ */
bloque_for ::=

  /* FOR clásico vacío */
  FOR PARENTESIS_ABRE
     VARIABLE:v IGUAL expresion:ini PUNTOCOMA
     condicion:cond PUNTOCOMA
     VARIABLE:vInc IGUAL expresion:inc
  PARENTESIS_CIERRA
  LLAVE_ABRE LLAVE_CIERRA
  {:
     parser.semantico.declararVariableFor(v.toString(), ini, 0, 0);
     RESULT = null;
  :}


  | FOR PARENTESIS_ABRE
       VARIABLE:v IGUAL expresion:ini PUNTOCOMA
       condicion:cond PUNTOCOMA
       VARIABLE:vInc IGUAL expresion:inc
    PARENTESIS_CIERRA
    LLAVE_ABRE lista_instrucciones_bloque:instr LLAVE_CIERRA
  {:
     parser.semantico.declararVariableFor(v.toString(), ini, 0, 0);
     int iter = 0;

     /* Primera evaluación usa cond (snapshot al parsear) */
     boolean condActual = parser.semantico.evaluarCondicion(cond);
     while (condActual && iter < parser.semantico.getMaxIteraciones()) {

        /* Ejecuta el cuerpo */
        for (Object o : (ArrayList<Object>) instr)
           if (o != null) ResultParser.currentForm.addElement(o);

        double nuevoValor = parser.semantico.evaluar(inc);
```

```
        parser.semantico.actualizarVariableCiclo(vInc.toString(), nuevoValor);

        iter++;

        /* Re-evalúa la condición con los valores actualizados */
        condActual = parser.semantico.recalcularUltimaCondicion();
    }
    parser.semantico.reportarLimiteCiclo("FOR", iter, 0, 0);
    RESULT = null;
:}

/* FOR con rango vacío */
| FOR PARENTESIS_ABRE
      VARIABLE:v IN expresion:desde RANGO expresion:hasta
  PARENTESIS_CIERRA
  LLAVE_ABRE LLAVE_CIERRA
{:
    parser.semantico.declararVariableFor(v.toString(), desde, 0, 0);
    RESULT = null;
:}


| FOR PARENTESIS_ABRE
      VARIABLE:v IN expresion:desde RANGO expresion:hasta
  PARENTESIS_CIERRA
  LLAVE_ABRE lista_instrucciones_bloque:instr LLAVE_CIERRA
{:
    int start = (int) parser.semantico.evaluar(desde);
    int end   = (int) parser.semantico.evaluar(hasta);

    /* Valida que el rango sea válido */
    if (start > end) {
        parser.semantico.getErrores().add(
            new com.example.proyecto1_compi1.token.Token(
                v.toString(), 0, 0, "Advertencia",
                "FOR rango: inicio (" + start + ") > fin (" + end +
                ") — el ciclo no se ejecutará."
            )
        );
    }

    parser.semantico.declararVariableFor(v.toString(), (double) start, 0, 0);

    for (int i = start; i <= end; i++) {
        /* Actualiza la variable ANTES de ejecutar el cuerpo */
        parser.semantico.actualizarVariableCiclo(v.toString(), (double) i);

        for (Object o : (ArrayList<Object>) instr)
            if (o != null) ResultParser.currentForm.addElement(o);
    }
    RESULT = null;
:}
;
```

```
/*
  PREGUNTAS
 */
open_q ::=
   OPENQUESTION CORCHETE_ABRE question_props:p CORCHETE_CIERRA
   {:
      OpenQuestion open = new OpenQuestion();
      for (PropertyItem prop : p) open.addProperty(prop);
      RESULT = open;
   :}
   ;

drop_q ::=
   DROPQUESTION CORCHETE_ABRE question_props:p CORCHETE_CIERRA
   {:
      DropQuestion drop = new DropQuestion();
      for (PropertyItem prop : p) drop.addProperty(prop);
      parser.semantico.validarCorrect(
         drop.getLabel(), drop.getOptions(), drop.getCorrect(), 0, 0);
      RESULT = drop;
   :}
   ;

select_q ::=
   SELECTQUESTION CORCHETE_ABRE question_props:p CORCHETE_CIERRA
   {:
      SelectQuestion select = new SelectQuestion();
      for (PropertyItem prop : p) select.addProperty(prop);
      parser.semantico.validarOpcionesSelect(
         select.getLabel(), select.getOptions(), 0, 0);
      parser.semantico.validarCorrect(
         select.getLabel(), select.getOptions(), select.getCorrect(), 0, 0);
      RESULT = select;
   :}
   ;

multiple_q ::=
   MULTIPLEQUESTION CORCHETE_ABRE question_props:p CORCHETE_CIERRA
   {:
      MultipleQuestion multiple = new MultipleQuestion();
      for (PropertyItem prop : p) multiple.addProperty(prop);
      parser.semantico.validarCorrect(
         multiple.getLabel(), multiple.getOptions(), multiple.getCorrect(), 0, 0);
      RESULT = multiple;
   :}
   ;

question_props ::=
   question_props:l COMA question_prop:p {: l.add(p); RESULT = l; :}
   | question_prop:p
   {:
      ArrayList<PropertyItem> list = new ArrayList<>();
```

```
        list.add(p);
        RESULT = list;
    :}
    ;

question_prop ::=
    LABEL DOSPUNTOS expresion:e
    {:
        RESULT = new PropertyItem("label",
            parser.semantico.expresionAString(e));
    :}
    | WIDTH DOSPUNTOS comodin_valor:n
    {: RESULT = new PropertyItem("width",  parser.semantico.evaluarEntero(n)); :}
    | HEIGHT DOSPUNTOS comodin_valor:n
    {: RESULT = new PropertyItem("height", parser.semantico.evaluarEntero(n)); :}
    | OPTIONS DOSPUNTOS lista_opciones:o
    {: RESULT = new PropertyItem("options", o); :}
    | OPTIONS DOSPUNTOS poke_call:p
    {: RESULT = new PropertyItem("options_poke", p); :}
    | CORRECT DOSPUNTOS expresion:n
    {:
        ArrayList<Integer> list = new ArrayList<>();
        list.add(parser.semantico.evaluarEntero(n));
        RESULT = new PropertyItem("correct", list);
    :}
    | CORRECT DOSPUNTOS LLAVE_ABRE int_list:l LLAVE_CIERRA
    {: RESULT = new PropertyItem("correct", l); :}
    | CORRECT DOSPUNTOS LLAVE_ABRE LLAVE_CIERRA
    {: RESULT = new PropertyItem("correct", new ArrayList<Integer>()); :}
    | styles_block:s
    {: RESULT = new PropertyItem("styles", s); :}
    ;

comodin_valor ::=
    expresion:e     {: RESULT = e; :}
    ;

lista_opciones ::=
    LLAVE_ABRE lista_strings:l LLAVE_CIERRA {: RESULT = l; :}
    ;



/* ── 1. lista_strings — acepta CADENA, comodín y expresión ──────────────── */
lista_strings ::=

    /* Base: un CADENA */
    CADENA:c
    {:
        ArrayList<String> list = new ArrayList<>();
        list.add(c.toString().trim());
        RESULT = list;
    :}
```

```
  /* Base: un comodín */
  | INTERROGACION
  {:
      ArrayList<String> list = new ArrayList<>();
      list.add("?");
      RESULT = list;
  :}

  /* Acumulación: + CADENA */
  | lista_strings:l COMA CADENA:c
  {:
      l.add(c.toString().trim());
      RESULT = l;
  :}

  /* Acumulación: + comodín */
  | lista_strings:l COMA INTERROGACION
  {:
      l.add("?");
      RESULT = l;
  :}
  ;

/* ── 2. int_list — acepta expresion y comodín, no solo ENTERO ──────────────── */
int_list ::=

  /* Base: expresión numérica (cubre ENTERO, DECIMAL, variable) */
  expresion:e
  {:
      ArrayList<Integer> list = new ArrayList<>();
      list.add(parser.semantico.evaluarEntero(e));
      RESULT = list;
  :}


  /* Acumulación: + expresión */
  | int_list:l COMA expresion:e
  {:
      l.add(parser.semantico.evaluarEntero(e));
      RESULT = l;
  :}

  ;

/*
  TEXTO
  */
text_decl ::=
  TEXT CORCHETE_ABRE text_props:p CORCHETE_CIERRA
  {:
      TextModel text = new TextModel();
      for (PropertyItem prop : p) text.addProperty(prop);
```

```
        RESULT = text;
    :}
    ;

text_props ::=
    text_props:l COMA text_prop:p {: l.add(p); RESULT = l; :}
    | text_prop:p
    {:
        ArrayList<PropertyItem> list = new ArrayList<>();
        list.add(p);
        RESULT = list;
    :}
    ;

text_prop ::=
    CONTENT DOSPUNTOS expresion:e
    {:
        RESULT = new PropertyItem("content",
            parser.semantico.expresionAString(e));
    :}
    | LABEL DOSPUNTOS expresion:e
    {:
        RESULT = new PropertyItem("label",
            parser.semantico.expresionAString(e));
    :}
    | WIDTH  DOSPUNTOS expresion:n
    {: RESULT = new PropertyItem("width",  parser.semantico.evaluarEntero(n)); :}
    | HEIGHT DOSPUNTOS expresion:n
    {: RESULT = new PropertyItem("height", parser.semantico.evaluarEntero(n)); :}
    | styles_block:s
    {: RESULT = new PropertyItem("styles", s); :}
    ;

/* ============================================================
   SECCIONES
   */
section ::=
    SECTION CORCHETE_ABRE section_props:p CORCHETE_CIERRA
    {: RESULT = new SectionsModel(p); :}
    ;

section_props ::=
    section_props:l COMA section_prop:p {: l.add(p); RESULT = l; :}
    | section_prop:p
    {:
        ArrayList list = new ArrayList();
        list.add(p);
        RESULT = list;
    :}
    ;

section_prop ::=
    WIDTH  DOSPUNTOS expresion:n
```

```
    {: RESULT = new PropertyItem("width", (int) parser.semantico.evaluar(n)); :}
    | HEIGHT DOSPUNTOS expresion:n
    {: RESULT = new PropertyItem("height", (int) parser.semantico.evaluar(n)); :}
    | POINTX DOSPUNTOS expresion:n
    {: RESULT = new PropertyItem("pointX", (int) parser.semantico.evaluar(n)); :}
    | POINTY DOSPUNTOS expresion:n
    {: RESULT = new PropertyItem("pointY", (int) parser.semantico.evaluar(n)); :}
    | ORIENTATION DOSPUNTOS VERTICAL
    {: RESULT = new PropertyItem("orientation", "VERTICAL"); :}
    | ORIENTATION DOSPUNTOS HORIZONTAL
    {: RESULT = new PropertyItem("orientation", "HORIZONTAL"); :}
    | ELEMENTS DOSPUNTOS LLAVE_ABRE element_list:e LLAVE_CIERRA
    {: RESULT = new PropertyItem("elements", e); :}
    | styles_block:s
    {: RESULT = new PropertyItem("styles", s); :}
    ;

element_list ::=
    element_list:l COMA element_en_lista:e
    {:
        if (e != null) l.add(e);
        RESULT = l;
    :}
    | element_en_lista:e
    {:
        ArrayList list = new ArrayList();
        if (e != null) list.add(e);
        RESULT = list;
    :}
    ;

element_en_lista ::=
    element:e      {: RESULT = e; :}
    | draw_stmt:d    {: RESULT = d; :}
    ;
/* ============================================================
   TABLAS
   ============================================================ */
table ::=
    TABLE CORCHETE_ABRE table_props:tp CORCHETE_CIERRA
    {: RESULT = new TableModel(tp); :}
    ;

table_props ::=
    table_props:l COMA table_prop:p {: l.add(p); RESULT = l; :}
    | table_prop:p
    {:
        ArrayList list = new ArrayList();
        list.add(p);
        RESULT = list;
    :}
    ;
```

```
table_prop ::=
   WIDTH  DOSPUNTOS expresion:n {: RESULT = new PropertyItem("width",  (int)
parser.semantico.evaluar(n)); :}
   | HEIGHT DOSPUNTOS expresion:n {: RESULT = new PropertyItem("height", (int)
parser.semantico.evaluar(n)); :}
   | POINTX DOSPUNTOS expresion:n {: RESULT = new PropertyItem("pointX", (int)
parser.semantico.evaluar(n)); :}
   | POINTY DOSPUNTOS expresion:n {: RESULT = new PropertyItem("pointY", (int)
parser.semantico.evaluar(n)); :}
   | ELEMENTS DOSPUNTOS LLAVE_ABRE table_rows:r LLAVE_CIERRA
   {: RESULT = new PropertyItem("elements", r); :}
   | styles_block:s
   {: RESULT = new PropertyItem("styles", s); :}
   ;

table_rows ::=
   table_rows:l COMA table_row:r {: l.add(r); RESULT = l; :}
   | table_row:r
   {:
      ArrayList list = new ArrayList();
      list.add(r);
      RESULT = list;
   :}
   ;

table_row ::=
   CORCHETE_ABRE table_cells:c CORCHETE_CIERRA {: RESULT = c; :}
   ;

table_cells ::=
   table_cells:l COMA LLAVE_ABRE element_en_lista:e LLAVE_CIERRA
   {: if (e != null) l.add(e); RESULT = l; :}
   | LLAVE_ABRE element_en_lista:e LLAVE_CIERRA
   {:
      ArrayList list = new ArrayList();
      if (e != null) list.add(e);
      RESULT = list;
   :}
   ;

/* ============================================================
  VARIABLE SPECIAL Y DRAW
  ============================================================ */
tipo_pregunta ::=
   OPENQUESTION    {: RESULT = "OPENQUESTION";    :}
   | DROPQUESTION  {: RESULT = "DROPQUESTION";    :}
   | SELECTQUESTION  {: RESULT = "SELECTQUESTION";  :}
   | MULTIPLEQUESTION {: RESULT = "MULTIPLEQUESTION"; :}
   ;

special_decl ::=
   SPECIAL VARIABLE:v IGUAL tipo_pregunta:t
   CORCHETE_ABRE special_props:props CORCHETE_CIERRA
```

```
    {:
        ArrayList<Integer> wildcards = new ArrayList<>();
        int idx = 0;
        for (PropertyItem item : props) {
            if (contieneComodin(item.value)) wildcards.add(idx);
            idx++;
        }
        SpecialQuestion sq = new SpecialQuestion(v.toString(), t, props, wildcards);
        if (ResultParser.currentForm != null)
            ResultParser.currentForm.addSpecialQuestion(sq);
        RESULT = sq;
    :}
    ;

special_props ::=
    special_props:l COMA special_prop:p {: l.add(p); RESULT = l; :}
    | special_prop:p
    {:
        ArrayList<PropertyItem> lst = new ArrayList<>();
        lst.add(p);
        RESULT = lst;
    :}
    ;

special_prop ::=
    WIDTH  DOSPUNTOS special_prop_value:v {: RESULT = new PropertyItem("width",  v); :}
    | HEIGHT DOSPUNTOS special_prop_value:v {: RESULT = new PropertyItem("height", v); :}
    | LABEL  DOSPUNTOS special_prop_value:v {: RESULT = new PropertyItem("label",  v); :}
    | OPTIONS DOSPUNTOS lista_opciones:o    {: RESULT = new PropertyItem("options", o); :}
    | OPTIONS DOSPUNTOS poke_call:p         {: RESULT = new PropertyItem("options_poke", p); :}
    | CORRECT DOSPUNTOS special_prop_value:v {: RESULT = new PropertyItem("correct", v); :}
    | CORRECT DOSPUNTOS LLAVE_ABRE int_list:l LLAVE_CIERRA
    {: RESULT = new PropertyItem("correct", l); :}
    | CORRECT DOSPUNTOS LLAVE_ABRE LLAVE_CIERRA
    {: RESULT = new PropertyItem("correct", new ArrayList<Integer>()); :}
    | styles_block:s
    {: RESULT = new PropertyItem("styles", s); :}
    | VARIABLE:key DOSPUNTOS special_prop_value:v
    {: RESULT = new PropertyItem(key.toString().toLowerCase(), v); :}
    ;

special_prop_value ::=
    expresion:e     {: RESULT = e; :}
    ;

draw_stmt ::=
    VARIABLE:v PUNTO DRAW
    PARENTESIS_ABRE parametros:args PARENTESIS_CIERRA
    {:
        SpecialQuestion sq = ResultParser.currentForm != null
            ? ResultParser.currentForm.findSpecialQuestion(v.toString()) : null;
        if (sq == null) {
            parser.semantico.getErrores().add(new Token(
```

```
            v.toString(), 0, 0, "Semántico",
            "Variable special '" + v + "' no declarada."));
        } else {
            // EVALUAR TODOS LOS ARGUMENTOS ANTES DE PASARLOS
            ArrayList<Object> evaluatedArgs = new ArrayList<>();
            for (Object arg : args) {
                Object evaluated = evaluarArgumento(arg);
                evaluatedArgs.add(evaluated);
                System.out.println("[draw] Argumento original: " + arg +
                                   " -> evaluado: " + evaluated);
            }

            if (evaluatedArgs.size() != sq.getWildcardCount()) {
                parser.semantico.getErrores().add(new Token(
                    v.toString(), 0, 0, "Semántico",
                    "'" + v + ".draw()' espera " + sq.getWildcardCount() +
                    " argumento(s) pero recibió " + evaluatedArgs.size() + "."));
            } else {
                sq.setDrawArguments(evaluatedArgs);
                QuestionModel q = sq.draw();
                if (q != null && ResultParser.currentForm != null)
                    ResultParser.currentForm.addElement(q);
            }
        }
    :}

    | VARIABLE:v PUNTO DRAW PARENTESIS_ABRE PARENTESIS_CIERRA
    {:
        SpecialQuestion sq = ResultParser.currentForm != null
            ? ResultParser.currentForm.findSpecialQuestion(v.toString()) : null;
        if (sq == null) {
            parser.semantico.getErrores().add(new Token(
                v.toString(), 0, 0, "Semántico",
                "Variable special '" + v + "' no declarada."));
        } else if (sq.getWildcardCount() != 0) {
            parser.semantico.getErrores().add(new Token(
                v.toString(), 0, 0, "Semántico",
                "'" + v + ".draw()' necesita " + sq.getWildcardCount() +
                " argumento(s)."));
        } else {
            QuestionModel q = sq.draw();
            if (q != null && ResultParser.currentForm != null)
                ResultParser.currentForm.addElement(q);
        }
    :}
    ;

parametros ::=
    parametros:p COMA expresion:e
    {: p.add(e); RESULT = p; :}
    | expresion:e
    {:
        ArrayList<Object> list = new ArrayList<>();
```

```
        list.add(e);
        RESULT = list;
    :}
    ;

/* POKE API */
poke_call ::=
    WHO_IS_THAT_POKEMON
    PARENTESIS_ABRE expresion:tipo COMA expresion:desde COMA expresion:hasta
    PARENTESIS_CIERRA
    {:
        ArrayList<String> params = new ArrayList<>();
        params.add(tipo.toString());
        params.add("?".equals(desde) ? "?" :
            String.valueOf(parser.semantico.evaluarEntero(desde)));
        params.add("?".equals(hasta) ? "?" :
            String.valueOf(parser.semantico.evaluarEntero(hasta)));
        RESULT = params;
    :}
    ;

/* ESTILOS Y COLORES */
styles_block ::=
    STYLES CORCHETE_ABRE style_list:s CORCHETE_CIERRA {: RESULT = s; :}
    ;

style_list ::=
    style_list:l COMA style_item:s {: l.add(s); RESULT = l; :}
    | style_item:s
    {:
        ArrayList<Object> list = new ArrayList<>();
        list.add(s);
        RESULT = list;
    :}
    ;

style_item ::=
    STYLE_COLOR DOSPUNTOS color_value:c
    {: RESULT = new ColorStyle(c); :}

    | STYLE_BACKGROUND DOSPUNTOS color_value:c
    {: RESULT = new BackgroundStyle(c); :}

    | STYLE_SIZE DOSPUNTOS expresion:n
    {: RESULT = new TextSizeStyle(parser.semantico.evaluarEntero(n)); :}

    | STYLE_FONT DOSPUNTOS font_value:f
    {: RESULT = new FontStyle(f); :}

    /* "border": (grosor, TIPO, color) */
    | STYLE_BORDER DOSPUNTOS PARENTESIS_ABRE
        expresion:grosor COMA tipo_borde:t COMA color_value:c
      PARENTESIS_CIERRA
```

```
    {:
        RESULT = t + ":" + parser.semantico.evaluarEntero(grosor) +
                ":" + c.toString();
    :}

    /* RGB con comodines: "color": (r, g, b) */
    | STYLE_COLOR DOSPUNTOS PARENTESIS_ABRE
        expresion:r COMA expresion:g COMA expresion:b
      PARENTESIS_CIERRA
    {:
        RESULT = new RgbStyleWithWildcard(r, g, b);
    :}

    | STYLE_BACKGROUND DOSPUNTOS PARENTESIS_ABRE
        expresion:r COMA expresion:g COMA expresion:b
      PARENTESIS_CIERRA
    {:
        RESULT = new BackgroundStyleWithWildcard(r, g, b);
    :}

    /* HSL con comodines: "color": <h, s, l> */
    | STYLE_COLOR DOSPUNTOS MENOR
        expresion:r COMA expresion:g COMA expresion:b
      MAYOR
    {:
        RESULT = new RgbStyleWithWildcard(r, g, b);
    :}

    | STYLE_BACKGROUND DOSPUNTOS MENOR
        expresion:r COMA expresion:g COMA expresion:b
      MAYOR
    {:
        RESULT = new BackgroundStyleWithWildcard(r, g, b);
    :}
    ;

comodin_color ::=
    expresion:e     {: RESULT = e; :}
;

tipo_borde ::=
    SOLID   {: RESULT = "SOLID";  :}
    | DOUBLE  {: RESULT = "DOUBLE"; :}
    | DOTTED  {: RESULT = "DOTTED"; :}
    | LINE    {: RESULT = "LINE"; :}
    ;
color_value ::=
    HEXCOLOR:h
    {: RESULT = new HexaColor(h.toString()); :}

    | RGB_COLOR:r
    {:
        String s = r.toString().replaceAll("[()\\s]", "");
```

```
        String[] p = s.split(",");
        RESULT = new RgbColor(
            Integer.parseInt(p[0].trim()),
            Integer.parseInt(p[1].trim()),
            Integer.parseInt(p[2].trim())
        );
    :}

    | HSL_COLOR:h
    {:
        String s = h.toString().replaceAll("[<>\\s]", "");
        String[] p = s.split(",");
        RESULT = new HslColor(
            Float.parseFloat(p[0].trim()),
            Float.parseFloat(p[1].trim()),
            Float.parseFloat(p[2].trim())
        );
    :}

    | RED    {: RESULT = BaseColor.RED;    :}
    | BLUE   {: RESULT = BaseColor.BLUE;   :}
    | GREEN  {: RESULT = BaseColor.GREEN;  :}
    | PURPLE {: RESULT = BaseColor.PURPLE; :}
    | SKY    {: RESULT = BaseColor.SKY;    :}
    | YELLOW {: RESULT = BaseColor.YELLOW; :}
    | BLACK  {: RESULT = BaseColor.BLACK;  :}
    | WHITE  {: RESULT = BaseColor.WHITE;  :}
    ;

font_value ::=
    MONO       {: RESULT = "MONO";      :}
    | SANS_SERIF  {: RESULT = "SANS_SERIF"; :}
    | CURSIVE     {: RESULT = "CURSIVE";    :}
    ;
```

PARSER .PKM

```
/* ── Terminales ── */
terminal METADATA_BLOCK;
terminal SECTION_OPEN, SECTION_CLOSE;
terminal CONTENT_OPEN, CONTENT_CLOSE;
terminal TEXT_ELEMENT;
terminal OPEN_TAG,     OPEN_CLOSE;
terminal SELECT_TAG,   SELECT_CLOSE;
terminal MULTIPLE_TAG, MULTIPLE_CLOSE;
terminal DROP_TAG,     DROP_CLOSE;
terminal TABLE_OPEN,   TABLE_CLOSE;
terminal LINE_OPEN,    LINE_CLOSE;
terminal ELEMENT_OPEN, ELEMENT_CLOSE;
terminal STYLE_OPEN,   STYLE_CLOSE;
terminal STYLE_COLOR, STYLE_BG, STYLE_FONT, STYLE_TEXT, STYLE_BORDER;
terminal FAMILY_EQ, SIZE_EQ, COLOR_EQ;
terminal MONO, SANS_SERIF, CURSIVE;
terminal SOLID, DASHED, DOTTED;
terminal COMMA, COLON, GUION;
terminal LBRACE, RBRACE;
terminal IGUAL, MAYOR_QUE, MENOR_QUE;
terminal SLASH_CLOSE;
terminal String  STRING;
terminal Integer NUMBER;
terminal String  ID;
terminal String  HEX_COLOR;
terminal String  RGB_COLOR;
terminal String  HSL_COLOR;
terminal String  COLOR_NAME;
terminal TEXT_CLOSE, INTEABRE, INTECIERRA;

/* ── No terminales ── */
non terminal Object              pkm_file;
non terminal Object              elementos_principales;
non terminal Object              elemento_principal;
non terminal SectionsModel       section;
non terminal Object              content;
non terminal Object              element;
non terminal TextModel           texto_element;
non terminal Object              table_elem;
non terminal Object              table_content;
non terminal Object              table_line;
non terminal Object              table_lines;
non terminal Object              table_cell;
non terminal Object              table_cells;
non terminal OpenQuestion        open_question;
non terminal SelectQuestion      select_question;
non terminal MultipleQuestion    multiple_question;
non terminal DropQuestion        drop_question;
non terminal ArrayList<String>   options;
non terminal ArrayList<Integer>  int_list;
```

```
non terminal ArrayList<Integer>      int_list_opt;
non terminal ArrayList<PropertyItem> parametros_pregunta;
non terminal ArrayList<Object>       style_block;
non terminal Object                  style_item;
non terminal String                  color_value;
non terminal String                  font_value;
non terminal String                  border_type;
non terminal ArrayList<Object> style_item_list;

precedence left COMMA;
precedence nonassoc NUMBER;

/* ============================================================
  INICIO
  ============================================================ */
pkm_file ::=
   METADATA_BLOCK elementos_principales
   | elementos_principales
   ;

elementos_principales ::=
   elementos_principales elemento_principal
   | elemento_principal
   ;

elemento_principal ::=
   section:s
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(s);
   :}
   | table_elem:t
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(t);
   :}
   | open_question:q
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(q);
   :}
   | select_question:q
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(q);
   :}
   | multiple_question:q
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(q);
   :}
   | drop_question:q
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(q);
   :}
   | texto_element:tx
   {:
      if (ResultParser.currentForm != null) ResultParser.currentForm.addElement(tx);
```

```
  :}
  ;

/* ═══════════════════════════════════════════════
  SECCIÓN — puede tener estilos opcionales
  ═══════════════════════════════════════════════ */
section ::=

  /* Con estilos */
  SECTION_OPEN
    NUMBER:w COMMA NUMBER:h COMMA NUMBER:x COMMA NUMBER:y COMMA ID:id
  MAYOR_QUE
    style_block:st
    CONTENT_OPEN content:c CONTENT_CLOSE
  SECTION_CLOSE
  {:
    ArrayList<PropertyItem> props = buildSectionProps(w, h, x, y, id.toString(), c);
    props.add(new PropertyItem("styles", st));
    RESULT = new SectionsModel(props);
  :}

  /* Sin estilos */
  | SECTION_OPEN
    NUMBER:w COMMA NUMBER:h COMMA NUMBER:x COMMA NUMBER:y COMMA ID:id
   MAYOR_QUE
    CONTENT_OPEN content:c CONTENT_CLOSE
   SECTION_CLOSE
  {:
    RESULT = new SectionsModel(buildSectionProps(w, h, x, y, id.toString(), c));
  :}
  ;
/* ═══════════════════════════════════════════════
  CONTENT — cualquier elemento puede anidarse
  ═══════════════════════════════════════════════ */
content ::=
  content:c element:e
  {:
    ((ArrayList<Object>) c).add(e);
    RESULT = c;
  :}
  | element:e
  {:
    ArrayList<Object> list = new ArrayList<>();
    list.add(e);
    RESULT = list;
  :}
  ;

/*
 * element — todo lo que puede ir dentro de una sección o tabla:
 * texto, preguntas, sección anidada, tabla anidada
 */
element ::=
```

```
  texto_element:t      {: RESULT = t; :}
  | open_question:q   {: RESULT = q; :}
  | select_question:q {: RESULT = q; :}
  | drop_question:q   {: RESULT = q; :}
  | multiple_question:q {: RESULT = q; :}
  | section:s         {: RESULT = s; :}
  | table_elem:t      {: RESULT = t; :}
  ;

/* ═══════════════════════════════════════════════════
  TEXT ELEMENT
  Formas:
  1. <text=w,h,"contenido"/>
  2. <text=w,h,"contenido">
       <style>...</style>
     </text>
  ═══════════════════════════════════════════════════ */
texto_element ::=

  /* Forma corta: <text=w,h,"label"/> */
  TEXT_ELEMENT NUMBER:w COMMA NUMBER:h COMMA STRING:s SLASH_CLOSE
  {:
      TextModel text = new TextModel();
      text.addProperty(new PropertyItem("width",  w));
      text.addProperty(new PropertyItem("height", h));
      text.addProperty(new PropertyItem("label",  s.toString().trim()));
      RESULT = text;
  :}

  /* Forma larga con estilo: <text=w,h,"label">\n<style>...</style>\n</text> */
  | TEXT_ELEMENT NUMBER:w COMMA NUMBER:h COMMA STRING:s MAYOR_QUE
    style_block:st
    TEXT_CLOSE
  {:
      TextModel text = new TextModel();
      text.addProperty(new PropertyItem("width",  w));
      text.addProperty(new PropertyItem("height", h));
      text.addProperty(new PropertyItem("label",  s.toString().trim()));
      text.addProperty(new PropertyItem("styles", st));
      RESULT = text;
  :}

  /* Forma larga sin estilo: <text=w,h,"label">\n</text> */
  | TEXT_ELEMENT NUMBER:w COMMA NUMBER:h COMMA STRING:s MAYOR_QUE
    TEXT_CLOSE
  {:
      TextModel text = new TextModel();
      text.addProperty(new PropertyItem("width",  w));
      text.addProperty(new PropertyItem("height", h));
      text.addProperty(new PropertyItem("label",  s.toString().trim()));
      RESULT = text;
  :}
  ;
```

```
/* ═══════════════════════════════════════════════════
  TABLA
  Formas:
  <table=w,h,x,y>
    <style>...</style>   ← opcional
    <content>
      <line>
        <element> ... </element>
      </line>
    </content>
  </table>
  ═══════════════════════════════════════════════════ */
table_elem ::=

  /* Con estilos */
  TABLE_OPEN NUMBER:w COMMA NUMBER:h COMMA NUMBER:x COMMA NUMBER:y MAYOR_QUE
      style_block:st
      CONTENT_OPEN table_lines:rows CONTENT_CLOSE
  TABLE_CLOSE
  {:
      ArrayList<PropertyItem> props = new ArrayList<>();
      props.add(new PropertyItem("width",    w));
      props.add(new PropertyItem("height",   h));
      props.add(new PropertyItem("pointX",   x));
      props.add(new PropertyItem("pointY",   y));
      props.add(new PropertyItem("styles",   st));
      props.add(new PropertyItem("elements", rows));
      RESULT = new TableModel(props);
  :}

  /* Sin estilos */
  | TABLE_OPEN NUMBER:w COMMA NUMBER:h COMMA NUMBER:x COMMA NUMBER:y MAYOR_QUE
      CONTENT_OPEN table_lines:rows CONTENT_CLOSE
    TABLE_CLOSE
  {:
      ArrayList<PropertyItem> props = new ArrayList<>();
      props.add(new PropertyItem("width",    w));
      props.add(new PropertyItem("height",   h));
      props.add(new PropertyItem("pointX",   x));
      props.add(new PropertyItem("pointY",   y));
      props.add(new PropertyItem("elements", rows));
      RESULT = new TableModel(props);
  :}
  ;

table_lines ::=
  table_lines:l table_line:r
  {:
      ((ArrayList<Object>) l).add(r);
      RESULT = l;
  :}
  | table_line:r
  {:
```

```
        ArrayList<Object> list = new ArrayList<>();
        list.add(r);
        RESULT = list;
    :}
    ;

table_line ::=
    LINE_OPEN table_cells:c LINE_CLOSE
    {: RESULT = c; :}
    | LINE_OPEN LINE_CLOSE
    {:
        RESULT = new ArrayList<Object>();
    :}
    ;

table_cells ::=
    table_cells:l table_cell:c
    {:
        ((ArrayList<Object>) l).add(c);
        RESULT = l;
    :}
    | table_cell:c
    {:
        ArrayList<Object> list = new ArrayList<>();
        list.add(c);
        RESULT = list;
    :}
    ;

table_cell ::=
    ELEMENT_OPEN element:e ELEMENT_CLOSE {: RESULT = e; :}
    | ELEMENT_OPEN ELEMENT_CLOSE        {: RESULT = null; :}
    ;

/* ============================================================
   ESTILOS
   ============================================================ */
style_block ::=
    STYLE_OPEN style_item_list:l STYLE_CLOSE
    {: RESULT = l; :}
    | STYLE_OPEN STYLE_CLOSE
    {:  RESULT = new ArrayList<Object>(); :}
    ;


style_item_list ::=
    style_item_list:l style_item:s
    {:
        l.add(s);
        RESULT = l;
    :}
    | style_item:s
    {:
```

```
        ArrayList<Object> list = new ArrayList<>();
        list.add(s);
        RESULT = list;
    :}
    ;

style_item ::=
    /* <color=#FFFFFF/> */
    STYLE_COLOR color_value:c SLASH_CLOSE
    {: RESULT = new ColorStyle(parseColor(c)); :}

    /* <background color=... /> */
    | STYLE_BG COLOR_EQ color_value:c SLASH_CLOSE
    {: RESULT = new BackgroundStyle(parseColor(c)); :}

    /* <font family=MONO/> */
    | STYLE_FONT FAMILY_EQ font_value:f SLASH_CLOSE
    {: RESULT = new FontStyle(f); :}

    /* <text size=16/>  — forma corta */
    | STYLE_TEXT NUMBER:n SLASH_CLOSE
    {: RESULT = new TextSizeStyle(n); :}

    /* <text size=16>   — forma larga con > */
    | STYLE_TEXT NUMBER:n MAYOR_QUE
    {: RESULT = new TextSizeStyle(n); :}

    /* <border,2,SOLID,color=#000000/> */
    | STYLE_BORDER NUMBER:n COMMA border_type:t COMMA COLOR_EQ color_value:c SLASH_CLOSE
    {: RESULT = t + ":" + n + ":" + c; :}
    ;


color_value ::=
    HEX_COLOR:h     {: RESULT = h; :}
    | RGB_COLOR:r   {: RESULT = r; :}
    | HSL_COLOR:h   {: RESULT = h; :}
    | COLOR_NAME:c  {: RESULT = c; :}
    ;

font_value ::=
    MONO       {: RESULT = "MONO";       :}
    | SANS_SERIF  {: RESULT = "SANS_SERIF"; :}
    | CURSIVE     {: RESULT = "CURSIVE";    :}
    ;

border_type ::=
    SOLID   {: RESULT = "SOLID";  :}
    | DASHED  {: RESULT = "DASHED"; :}
    | DOTTED  {: RESULT = "DOTTED"; :}
    ;

/* ═══════════════════════════════════════════════════
```

```
  PARÁMETROS PREGUNTA
  ═══════════════════════════════════════════════════════════ */
parametros_pregunta ::=
   NUMBER:w COMMA NUMBER:h COMMA
   {:
      ArrayList<PropertyItem> props = new ArrayList<>();
      props.add(new PropertyItem("width",  w));
      props.add(new PropertyItem("height", h));
      RESULT = props;
   :}
   ;

/* ═══════════════════════════════════════════════════════════
  OPEN QUESTION (con y sin estilo)
  ═══════════════════════════════════════════════════════════ */
open_question ::=
   OPEN_TAG parametros_pregunta:p STRING:s SLASH_CLOSE
   {:
      OpenQuestion open = new OpenQuestion();
      for (PropertyItem prop : p) open.addProperty(prop);
      open.addProperty(new PropertyItem("label", s.toString().trim()));
      RESULT = open;
   :}
   | OPEN_TAG parametros_pregunta:p STRING:s MAYOR_QUE
    style_block:st
    OPEN_CLOSE
   {:
      OpenQuestion open = new OpenQuestion();
      for (PropertyItem prop : p) open.addProperty(prop);
      open.addProperty(new PropertyItem("label",  s.toString().trim()));
      open.addProperty(new PropertyItem("styles", st));
      RESULT = open;
   :}
   | OPEN_TAG parametros_pregunta:p STRING:s OPEN_CLOSE
   {:
      OpenQuestion open = new OpenQuestion();
      for (PropertyItem prop : p) open.addProperty(prop);
      open.addProperty(new PropertyItem("label", s.toString().trim()));
      RESULT = open;
   :}
   ;

/* ═══════════════════════════════════════════════════════════
  SELECT QUESTION
  ═══════════════════════════════════════════════════════════ */
select_question ::=
   SELECT_TAG parametros_pregunta:p STRING:s COMMA
            LBRACE options:o RBRACE COMMA NUMBER:n SLASH_CLOSE
   {:
      SelectQuestion q = new SelectQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
```

```
      ArrayList<Integer> c = new ArrayList<>(); c.add(n);
      q.addProperty(new PropertyItem("correct", c));
      RESULT = q;
    :}
    | SELECT_TAG parametros_pregunta:p STRING:s COMMA
              LBRACE options:o RBRACE COMMA NUMBER:n MAYOR_QUE
              style_block:st SELECT_CLOSE
    {:
      SelectQuestion q = new SelectQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
      ArrayList<Integer> c = new ArrayList<>(); c.add(n);
      q.addProperty(new PropertyItem("correct", c));
      q.addProperty(new PropertyItem("styles",  st));
      RESULT = q;
    :}
    | SELECT_TAG parametros_pregunta:p STRING:s COMMA
              LBRACE options:o RBRACE COMMA NUMBER:n SELECT_CLOSE
    {:
      SelectQuestion q = new SelectQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
      ArrayList<Integer> c = new ArrayList<>(); c.add(n);
      q.addProperty(new PropertyItem("correct", c));
      RESULT = q;
    :}
    ;

/* ============================================================
   DROP QUESTION
   ============================================================ */
drop_question ::=
    DROP_TAG parametros_pregunta:p STRING:s COMMA
          LBRACE options:o RBRACE COMMA NUMBER:n SLASH_CLOSE
    {:
      DropQuestion q = new DropQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
      ArrayList<Integer> c = new ArrayList<>(); c.add(n);
      q.addProperty(new PropertyItem("correct", c));
      RESULT = q;
    :}
    | DROP_TAG parametros_pregunta:p STRING:s COMMA
              LBRACE options:o RBRACE COMMA NUMBER:n MAYOR_QUE
              style_block:st DROP_CLOSE
    {:
      DropQuestion q = new DropQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
```

```
      ArrayList<Integer> c = new ArrayList<>(); c.add(n);
      q.addProperty(new PropertyItem("correct", c));
      q.addProperty(new PropertyItem("styles",  st));
      RESULT = q;
   :}
   | DROP_TAG parametros_pregunta:p STRING:s COMMA
            LBRACE options:o RBRACE COMMA NUMBER:n DROP_CLOSE
   {:
      DropQuestion q = new DropQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
      ArrayList<Integer> c = new ArrayList<>(); c.add(n);
      q.addProperty(new PropertyItem("correct", c));
      RESULT = q;
   :}
   ;

/* ═══════════════════════════════════════════════════
  MULTIPLE QUESTION
  ═══════════════════════════════════════════════════ */
multiple_question ::=
   MULTIPLE_TAG parametros_pregunta:p STRING:s COMMA
            LBRACE options:o RBRACE COMMA
            LBRACE int_list:cl RBRACE SLASH_CLOSE
   {:
      MultipleQuestion q = new MultipleQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
      q.addProperty(new PropertyItem("correct", cl));
      RESULT = q;
   :}
   | MULTIPLE_TAG parametros_pregunta:p STRING:s COMMA
               LBRACE options:o RBRACE COMMA
               LBRACE RBRACE SLASH_CLOSE
   {:
      MultipleQuestion q = new MultipleQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
      q.addProperty(new PropertyItem("correct", new ArrayList<Integer>()));
      RESULT = q;
   :}
   | MULTIPLE_TAG parametros_pregunta:p STRING:s COMMA
               LBRACE options:o RBRACE COMMA
               LBRACE int_list:cl RBRACE MAYOR_QUE
               style_block:st MULTIPLE_CLOSE
   {:
      MultipleQuestion q = new MultipleQuestion();
      for (PropertyItem prop : p) q.addProperty(prop);
      q.addProperty(new PropertyItem("label",   s.toString().trim()));
      q.addProperty(new PropertyItem("options", o));
```

```
        q.addProperty(new PropertyItem("correct", cl));
        q.addProperty(new PropertyItem("styles",  st));
        RESULT = q;
    :}
    | MULTIPLE_TAG parametros_pregunta:p STRING:s COMMA
                LBRACE options:o RBRACE COMMA
                LBRACE int_list:cl RBRACE MULTIPLE_CLOSE
    {:
        MultipleQuestion q = new MultipleQuestion();
        for (PropertyItem prop : p) q.addProperty(prop);
        q.addProperty(new PropertyItem("label",   s.toString().trim()));
        q.addProperty(new PropertyItem("options", o));
        q.addProperty(new PropertyItem("correct", cl));
        RESULT = q;
    :}
    ;

/* ============================================================
  LISTAS
  ============================================================ */
options ::=
    STRING:s
    {:
        ArrayList<String> list = new ArrayList<>();
        list.add(s.toString().trim());
        RESULT = list;
    :}
    | options:o COMMA STRING:s
    {:
        o.add(s.toString().trim());
        RESULT = o;
    :}
    ;

int_list ::=
    NUMBER:n
    {:
        ArrayList<Integer> list = new ArrayList<>();
        list.add(n);
        RESULT = list;
    :}
    | int_list:l COMMA NUMBER:n
    {:
        l.add(n);
        RESULT = l;
    :}
    ;

int_list_opt ::=
    {: RESULT = new ArrayList<Integer>(); :}
    | int_list:l {: RESULT = l; :}
    ;
```

```
                                                    LEXER .PKM
/* ═══════════════════════════════════════════════════════════
  YYINITIAL
  ═══════════════════════════════════════════════════════════ */
<YYINITIAL> {

   "###"[^#]*"###"     { return symbol(sym.METADATA_BLOCK, yytext()); }

       /* ── Comentarios — SOLO al inicio de línea o con espacio antes ── */
       /* El # de hex color siempre viene después de = o espacio dentro de tag */
       "$"[^\r\n]*        { /* comentario $ */ }
       [ \t\r\n]+         { /* ignorar */ }

       /* ── Hex ANTES que cualquier otra regla  */
       {HEXCOLOR}         { return symbol(sym.HEX_COLOR, yytext()); }


   "</section>"       { return symbol(sym.SECTION_CLOSE);  }
   "</content>"       { return symbol(sym.CONTENT_CLOSE);  }
   "</table>"         { return symbol(sym.TABLE_CLOSE);    }
   "</line>"          { return symbol(sym.LINE_CLOSE);     }
   "</element>"       { return symbol(sym.ELEMENT_CLOSE);  }
   "</style>"         { return symbol(sym.STYLE_CLOSE);    }
   "</text>"          { return symbol(sym.TEXT_CLOSE);     }
   "</open>"          { return symbol(sym.OPEN_CLOSE);     }
   "</select>"        { return symbol(sym.SELECT_CLOSE);   }
   "</multiple>"      { return symbol(sym.MULTIPLE_CLOSE); }
   "</drop>"          { return symbol(sym.DROP_CLOSE);     }

   /*

       */

   /* Estilo texto — PRIMERO porque "<text size=" es más largo que "<text=" */
   "<text size="      { return symbol(sym.STYLE_TEXT);     }

   /* Apertura de texto — SEGUNDO */
   "<text="           { return symbol(sym.TEXT_ELEMENT);   }

   /* Sección */
   "<section="        { return symbol(sym.SECTION_OPEN);   }

   /* Tabla */
   "<table="          { return symbol(sym.TABLE_OPEN);     }

   /* Preguntas */
   "<open="           { return symbol(sym.OPEN_TAG);       }
   "<select="         { return symbol(sym.SELECT_TAG);     }
   "<multiple="       { return symbol(sym.MULTIPLE_TAG);   }
   "<drop="           { return symbol(sym.DROP_TAG);       }

   /* Estilos con = */
   "<color="          { return symbol(sym.STYLE_COLOR);    }
```

```
"<background"      { return symbol(sym.STYLE_BG);      }
"<font"            { return symbol(sym.STYLE_FONT);    }
"<border,"         { return symbol(sym.STYLE_BORDER);  }

/* — Aperturas sin = — */
"<content>"        { return symbol(sym.CONTENT_OPEN);  }
"<line>"           { return symbol(sym.LINE_OPEN);     }
"<element>"        { return symbol(sym.ELEMENT_OPEN);  }
"<style>"          { return symbol(sym.STYLE_OPEN);    }

/* — Atributos de estilo — */
"family="          { return symbol(sym.FAMILY_EQ);  }
"size="            { return symbol(sym.SIZE_EQ);    }
"color="           { return symbol(sym.COLOR_EQ);   }


"MONO"             { return symbol(sym.MONO);       }
"SANS_SERIF"       { return symbol(sym.SANS_SERIF); }
"CURSIVE"          { return symbol(sym.CURSIVE);    }
"SOLID"            { return symbol(sym.SOLID);      }
"DASHED"           { return symbol(sym.DASHED);     }
"DOTTED"           { return symbol(sym.DOTTED);     }

/* — Colores base — como COLOR_NAME — */
"RED"              { return symbol(sym.COLOR_NAME, "RED");    }
"BLUE"             { return symbol(sym.COLOR_NAME, "BLUE");   }
"GREEN"            { return symbol(sym.COLOR_NAME, "GREEN");  }
"PURPLE"           { return symbol(sym.COLOR_NAME, "PURPLE"); }
"SKY"              { return symbol(sym.COLOR_NAME, "SKY");    }
"YELLOW"           { return symbol(sym.COLOR_NAME, "YELLOW"); }
"BLACK"            { return symbol(sym.COLOR_NAME, "BLACK");  }
"WHITE"            { return symbol(sym.COLOR_NAME, "WHITE");  }


/* — RGB completo (10,10,10) — antes que "(" suelto — */
("{DIGIT}+","{DIGIT}+","{DIGIT}+")"
                   { return symbol(sym.RGB_COLOR, yytext()); }

/* — HSL completo <10,0.5,0.5> — antes que "<" suelto — */
"<"{DIGIT}+("."{DIGIT}+)?","{DIGIT}+("."{DIGIT}+)?","{DIGIT}+("."{DIGIT}+)?">"
                   { return symbol(sym.HSL_COLOR, yytext()); }

/* — Número negativo — ANTES que "-" suelto — */
"-"{DIGIT}+        { return symbol(sym.NUMBER, Integer.parseInt(yytext())); }

/* — Número positivo — */
{DIGIT}+           { return symbol(sym.NUMBER, Integer.parseInt(yytext())); }

/* — Cierres de tag — */
"/>"               { return symbol(sym.SLASH_CLOSE); }
">"                { return symbol(sym.MAYOR_QUE);   }

/* — Símbolos — */
```

```
  ","              { return symbol(sym.COMMA);  }
  ":"              { return symbol(sym.COLON);  }
  "-"              { return symbol(sym.GUION);  }
  "{"              { return symbol(sym.LBRACE); }
  "}"              { return symbol(sym.RBRACE); }
  "="              { return symbol(sym.IGUAL);  }

  /* — String — */
  \"               { stringBuffer.setLength(0); yybegin(STRING_STATE); }

  /* — ID — captura VERTICAL, HORIZONTAL y cualquier nombre — */
  {ID}             { return symbol(sym.ID, yytext()); }

  /* — Ignorar el resto — */
  .                { /* ignorar caracteres desconocidos */ }
}

/*
  STRING_STATE
  */
<STRING_STATE> {

  \"  {
      yybegin(YYINITIAL);
      return symbol(sym.STRING, stringBuffer.toString());
  }

  /* — Emojis — igual que antes — */
  "@[:smile:]"    { stringBuffer.append("😀"); }
  "@[:sad:]"      { stringBuffer.append("😢"); }
  "@[:serious:]"  { stringBuffer.append("😐"); }
  "@[:heart:]"    { stringBuffer.append("❤"); }
  "@[:cat:]"      { stringBuffer.append("😺"); }
  "@[:^^:]"       { stringBuffer.append("😺"); }
  "@[:star:]"     { stringBuffer.append("⭐"); }

  "@[:star:" {DIGIT}+ ":]"  {
      String txt = yytext();
      int n = Integer.parseInt(txt.substring(txt.indexOf("star:")+5, txt.lastIndexOf(":")));
      for (int i = 0; i < n; i++) stringBuffer.append("⭐");
  }
  "@[:star-" {DIGIT}+ "]"  {
      String txt = yytext();
      int n = Integer.parseInt(txt.substring(txt.indexOf("star-")+5, txt.lastIndexOf("]")));
      for (int i = 0; i < n; i++) stringBuffer.append("⭐");
  }
  "@[" "<"+ "3"+ "]"    { stringBuffer.append("❤"); }
  "@[:" ")"+ "]"        { stringBuffer.append("😀"); }
  "@[:" "("+ "]"        { stringBuffer.append("😢"); }
  "@[:" "|"+ "]"        { stringBuffer.append("😐"); }
  "@"                   { stringBuffer.append('@'); }

  /* — Escape — */
```

```
    "\\\""  { stringBuffer.append('"'); }
    "\\" .  { stringBuffer.append(yytext()); }

    /* ── Salto de línea — cierra el string ── */
    \n | \r | \r\n  {
        yybegin(YYINITIAL);
        return symbol(sym.STRING, stringBuffer.toString());
    }


    [^\"\\\n\r@]    { stringBuffer.append(yytext()); }
}
```